CÓDIGOCLARO · MATERIAL DE ESTUDIO

Desarrollo web: HTML, CSS y JavaScript

Cuaderno DAW · HTML Living Standard · CSS vigente · ECMAScript 2025 · APIs del navegador.

Estructura semántica

Objetivo: Organizar una página de prácticas para que se entienda sin depender de su apariencia.

HTML aporta significado y estructura. Usa títulos jerárquicos, una región principal, navegación con nombre y enlaces cuyo texto explique el destino. El atributo lang ayuda a las tecnologías de asistencia a pronunciar el contenido en español. Un elemento se elige por su función, no por el estilo predeterminado que muestra el navegador.

Código · HTML

```
<!doctype html>
<html lang="es">
<body>
  <header><h1>Prácticas DAM</h1></header>
  <nav aria-label="Principal"><a href="/modulos">Módulos</a></nav>
  <main><article><h2>Bases de datos</h2></article></main>
</body></html>
```

Práctica: Cambia el texto del enlace por “Pulsa aquí”. ¿Qué información pierde una persona que recorre enlaces fuera de contexto?

Respuesta razonada: Pierde la pista de que el destino es la lista de módulos. Un texto como “Consultar módulos” identifica mejor el enlace sin depender del texto que lo rodea.

Formularios y validación accesible

Objetivo: Recoger el correo del centro e indicar cómo corregir una entrada incorrecta.

Asocia cada control a una etiqueta visible y coloca las instrucciones cerca del campo. required y type=email ayudan a validar casos comunes en el navegador, pero no son una frontera de seguridad: la petición puede enviarse desde otro cliente. El servidor debe validar formato y regla de negocio de nuevo.

Código · HTML + WAI

```
<form>
  <label for="correo">Correo del centro</label>
  <input id="correo" name="correo" type="email" required>
  <button type="submit">Enviar solicitud</button>
</form>
```

Práctica: ¿Qué debe hacer el servidor si recibe una petición sin correo aunque la interfaz tenga required?

Respuesta razonada: Debe rechazarla con un mensaje comprensible. La validación cliente mejora el uso, pero se puede omitir; la etiqueta da nombre al control y el servidor vuelve a comprobar los datos.

Cascada, especificidad y modelo de caja

Objetivo: Corregir una tarjeta cuyo ancho no coincide con el diseño acordado.

La cascada considera origen, importancia, capas, especificidad y orden para elegir una declaración. El ancho se compone de contenido, padding, borde y margen; con border-box, padding y borde se incluyen en el tamaño declarado. Inspecciona estilos calculados antes de añadir reglas más específicas o !important.

Código · CSS Cascade

```
.tarjeta {
  box-sizing: border-box;
  width: 18rem;
  padding: 1rem;
  border: 1px solid #667085;
  color: #243b80;
}
```

Práctica: Dos reglas asignan color a .tarjeta. ¿Qué compruebas antes de aumentar la especificidad?

Respuesta razonada: Comprueba qué regla gana en el panel de estilos, su capa, importancia y orden. Elimina la regla duplicada si ya no tiene responsabilidad; aumentar especificidad suele crear una cadena difícil de mantener.

Flexbox, Grid y diseño adaptable

Objetivo: Distribuir tarjetas de módulos sin provocar desbordamiento en móvil.

Flexbox ordena elementos principalmente en una dimensión y Grid define filas y columnas. minmax y auto-fit ajustan columnas al espacio disponible. El diseño debe mantener el orden de lectura del documento y tolerar zoom, textos largos y pantallas estrechas; no añadas un breakpoint para cada anchura intermedia.

Código · CSS Grid

```
.catalogo {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(15rem, 1fr));
  gap: 1rem;
}
.tarjeta { min-width: 0; }
```

Práctica: Comprueba la cuadrícula a 320 px y 1280 px. ¿Cuántas columnas esperas en cada caso?

Respuesta razonada: En la ventana estrecha cabe una columna y no debe aparecer desplazamiento horizontal; en la ancha aparecen varias según el espacio. El contenido conserva su orden original.

Valores y comparaciones en JavaScript

Objetivo: Conservar un cero válido cuando se decide si quedan plazas disponibles.

JavaScript tiene valores falsy como 0, cadena vacía, false, null y undefined. El operador || devuelve uno de sus operandos y puede sustituir un cero válido; ?? solo recurre al valor alternativo ante null o undefined. === compara sin coerción implícita y hace visible la intención de la condición.

Código · ECMAScript 2025

```
const plazas = 0;
const conOr = plazas || 10;
const conNullish = plazas ?? 10;
console.log(conOr, conNullish); // 10, 0
```

Práctica: Si cero significa “completo”, ¿qué operador conserva el dato?

Respuesta razonada: ?? conserva cero porque no considera falsy todos los valores; || sustituye cero por 10. La regla debe expresar diferencia entre ausencia y valor numérico válido.

Transformaciones de arrays

Objetivo: Filtrar artículos caros y producir sus nombres sin alterar la lista fuente.

filter selecciona elementos y map transforma cada elemento a otro valor; ambos crean resultados nuevos y permiten leer el flujo de datos de izquierda a derecha. Define si el umbral es inclusivo y comprueba array vacío y límite. No compartas una mutación inesperada con otros componentes que usan los mismos datos.

Código · ECMAScript 2025

```
const productos = [
  { nombre: "Teclado", precio: 25 },
  { nombre: "Cable", precio: 8 }
];
const caros = productos.filter(p => p.precio > 20);
const nombres = caros.map(p => p.nombre);
```

Práctica: Cambia > por >= y fija el precio del Teclado en 20. ¿Qué cambia?

Respuesta razonada: La comparación estricta no selecciona el precio 20; la inclusiva sí. productos mantiene sus dos objetos porque filter construye una selección nueva.

DOM y eventos delegados

Objetivo: Manejar clics sobre tareas que se añaden después de cargar la página.

El DOM es un árbol de nodos y los eventos pueden propagarse hacia los ancestros. Un listener en el contenedor puede identificar el botón activado con closest y delegar el manejo de elementos dinámicos. Inserta texto de usuario con textContent; concatenarlo en innerHTML puede convertir contenido en marcado ejecutable.

Código · DOM Standard

```
const lista = document.querySelector("#tareas");
lista.addEventListener("click", evento => {
  const boton = evento.target.closest("button[data-id]");
  if (!boton || !lista.contains(boton)) return;
  console.log(boton.dataset.id);
});
```

Práctica: Añade una tarea después de registrar el listener. ¿Es necesario enlazarla individualmente?

Respuesta razonada: No. El clic sube hasta lista y el listener detecta el botón por su atributo. contains evita tratar un botón ajeno como hijo del catálogo.

Fetch, HTTP y validación de respuesta

Objetivo: Separar una respuesta HTTP de error de una respuesta válida con JSON inesperado.

fetch rechaza ante ciertos fallos de red, pero 404 o 500 son respuestas recibidas y requieren comprobar response.ok. Después de analizar JSON valida su forma: un documento correcto puede carecer de los campos esperados. Expón estados de carga y error separados para no presentar datos antiguos como nuevos.

Código · Fetch Standard

```
async function cargar(url) {
  const respuesta = await fetch(url);
  if (!respuesta.ok) throw new Error("HTTP " + respuesta.status);
  const datos = await respuesta.json();
  if (!Array.isArray(datos)) throw new Error("Formato");
  return datos;
}
```

Práctica: La respuesta es 200 y el cuerpo es un objeto, no una lista. ¿Qué debe hacer la función?

Respuesta razonada: Debe rechazar el formato o transformarlo según contrato; no puede tratar el objeto como una lista válida. Comprueba el estado HTTP y el esquema por separado.

Módulos y límites de ejecución

Objetivo: Reutilizar una función de precio tanto en navegador como en Node sin importar el DOM.

Los módulos declaran sus importaciones y exportaciones. La lógica de cálculo puede ser independiente del navegador; la capa de interfaz manipula document y la capa de servidor accede al sistema de archivos. Mantén esas dependencias en sus entornos para que el núcleo sea reutilizable y más fácil de probar.

Código · ECMAScript modules

```
// precios.js
export function conIva(base, tipo = 0.21) {
  return base * (1 + tipo);
}
// app.js
import { conIva } from "./precios.js";
console.log(conIva(100));
```

Práctica: ¿Por qué precios.js no debe consultar document si también se usa desde Node?

Respuesta razonada: document pertenece al navegador y no existe en Node. Separar el cálculo de la interfaz deja que ambos entornos compartan la misma regla sin crear globals simulados.

Teclado, foco y salida segura

Objetivo: Comprobar que una tarjeta es operable sin ratón y muestra literalmente el comentario recibido.

Una interfaz debe mantener foco visible y nombres accesibles en botones. El texto aportado por el usuario se debe insertar como texto, no como HTML. Prueba con teclado el orden de tabulación y una cadena que parezca una etiqueta; no uses aria-label para reemplazar una etiqueta visible que el usuario necesita.

Código · WAI + DOM

```
const comentario = document.querySelector("#comentario");
comentario.textContent = textoRecibido;
// Evitar asignar texto no confiable a innerHTML.
```

Práctica: Si textoRecibido es <img src=x onerror=alert(1)>, ¿qué debe ocurrir?

Respuesta razonada: Debe mostrarse la cadena literal y no crearse una imagen. textContent no interpreta el texto como HTML; la validación del servidor sigue siendo necesaria para los datos.

Fuentes oficiales

WHATWG HTML Living Standard · W3C CSS Cascade Level 5 · W3C Flexbox Level 1 · W3C Grid Level 2 · ECMAScript 2025 · W3C WAI: formularios · WHATWG Fetch Standard